



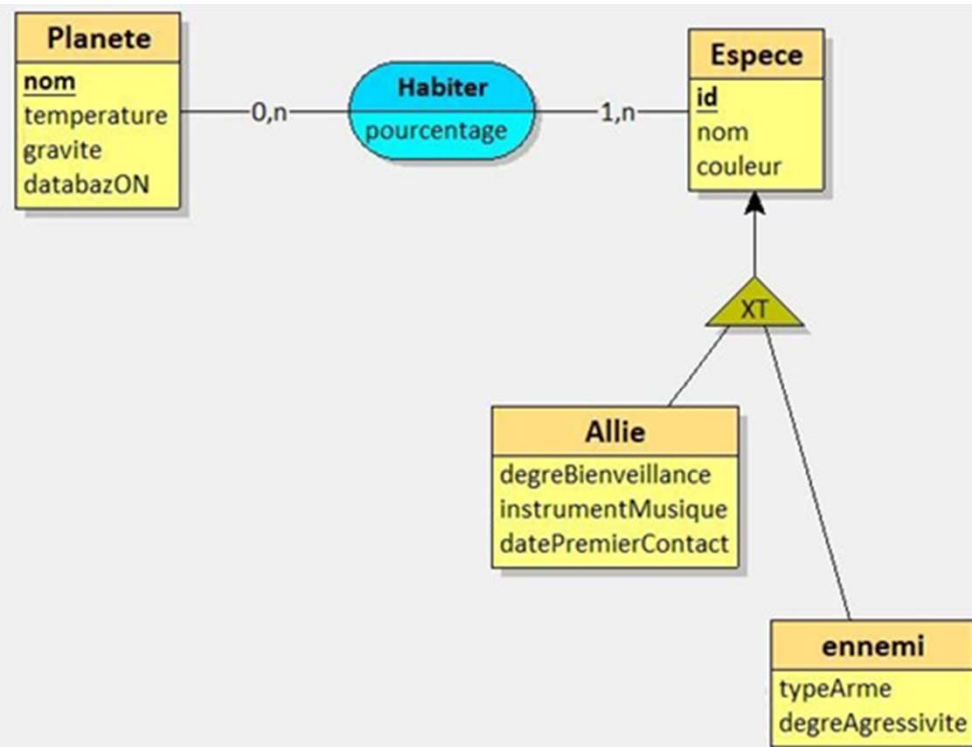
Planete

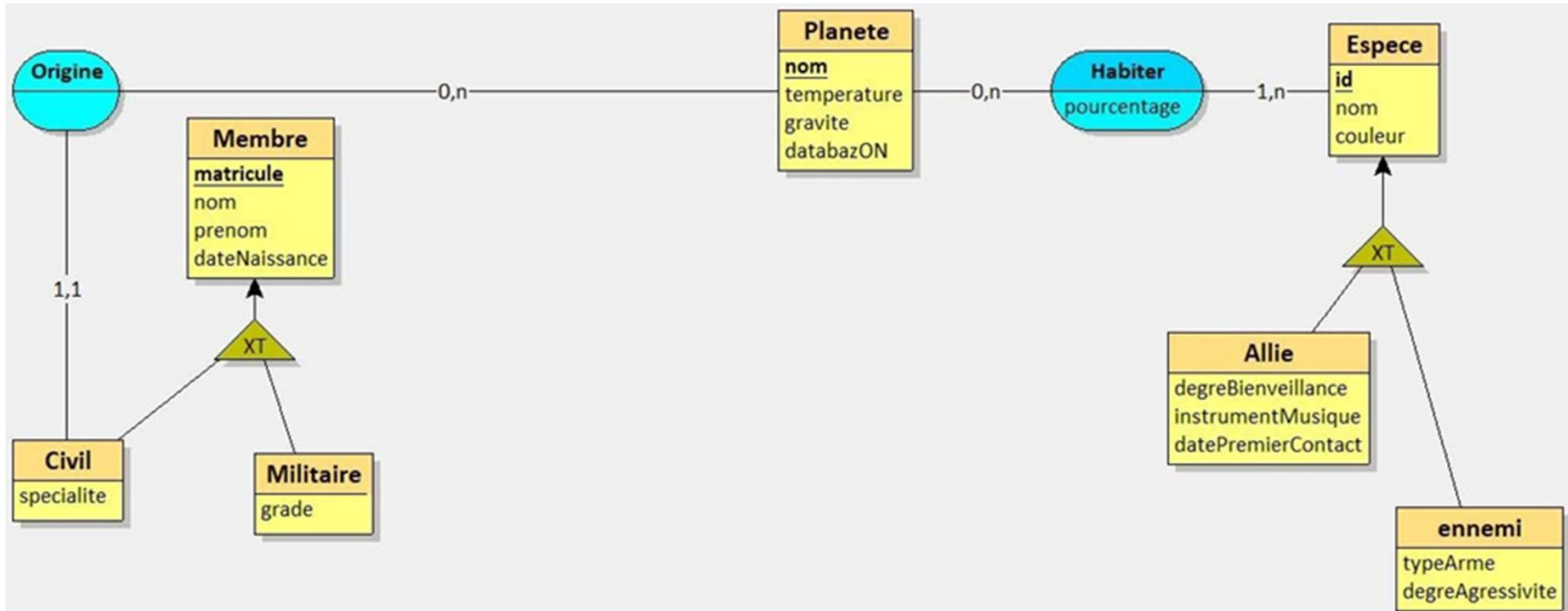
nom

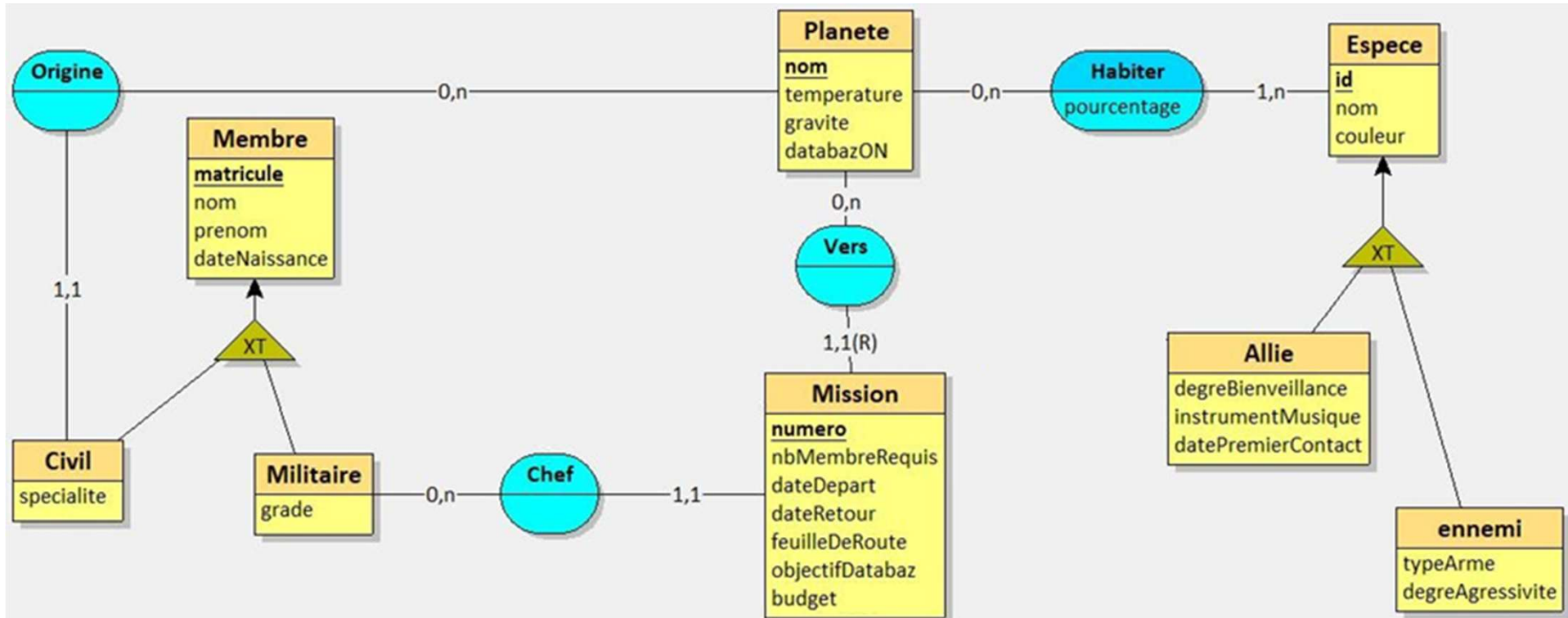
temperature

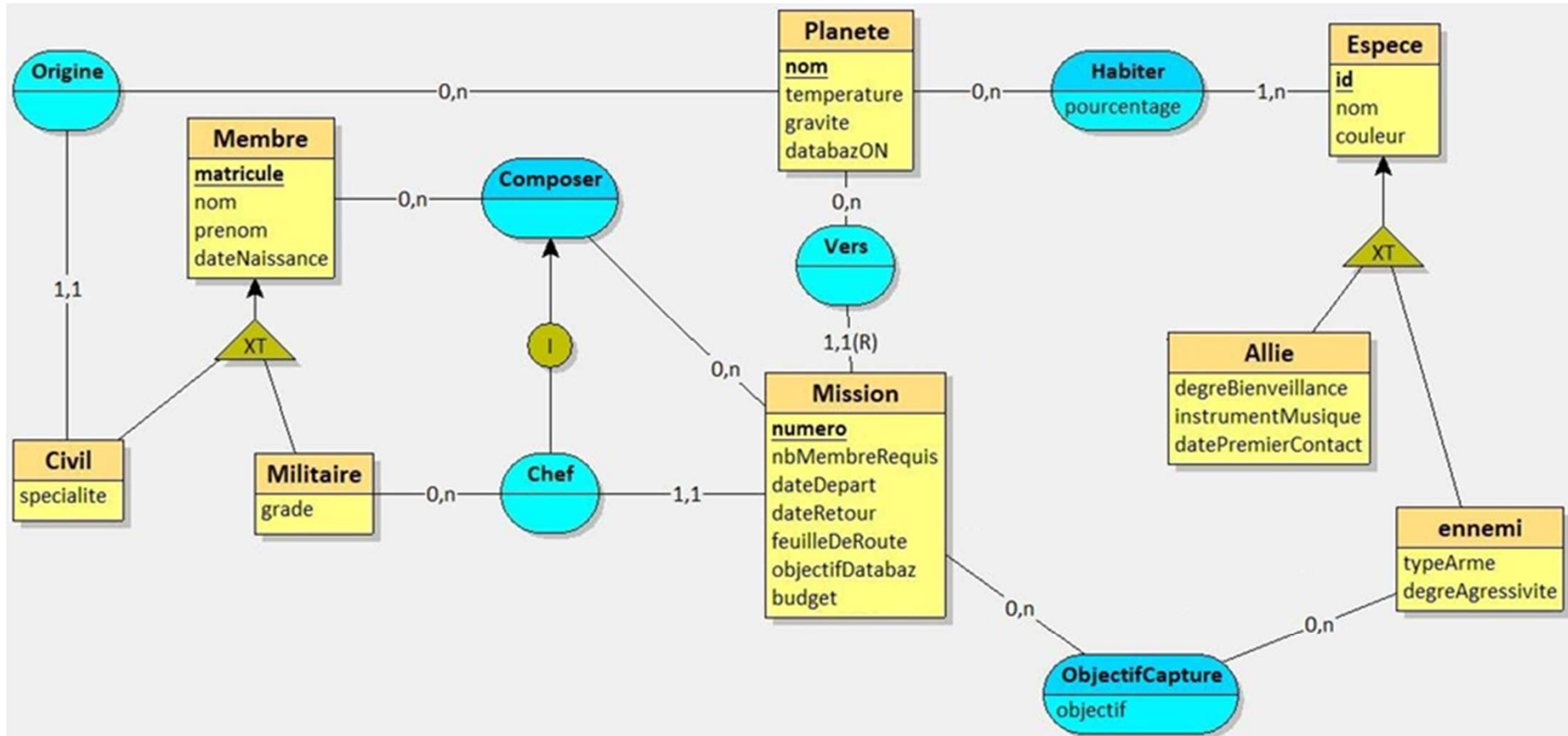
gravite

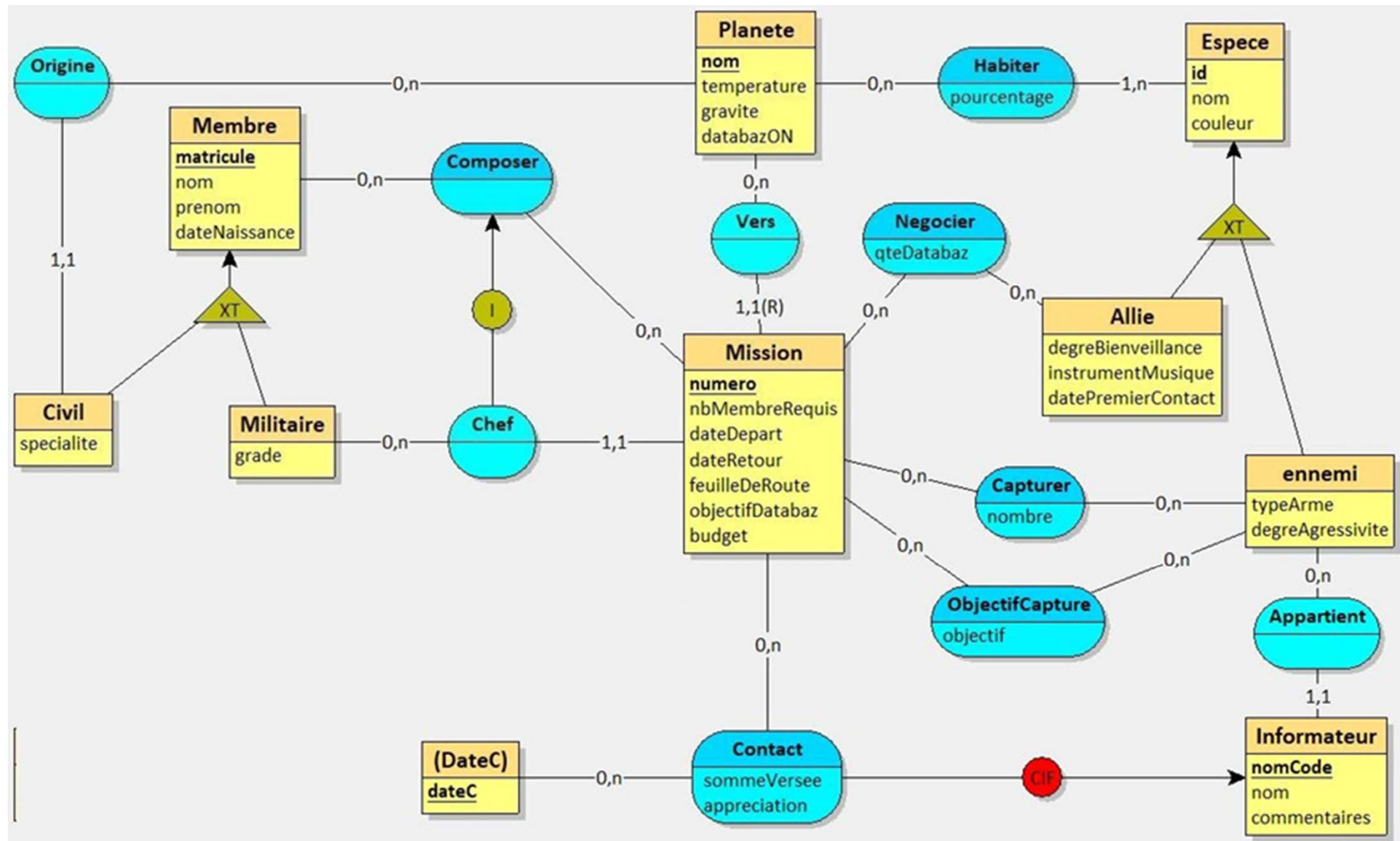
databazON

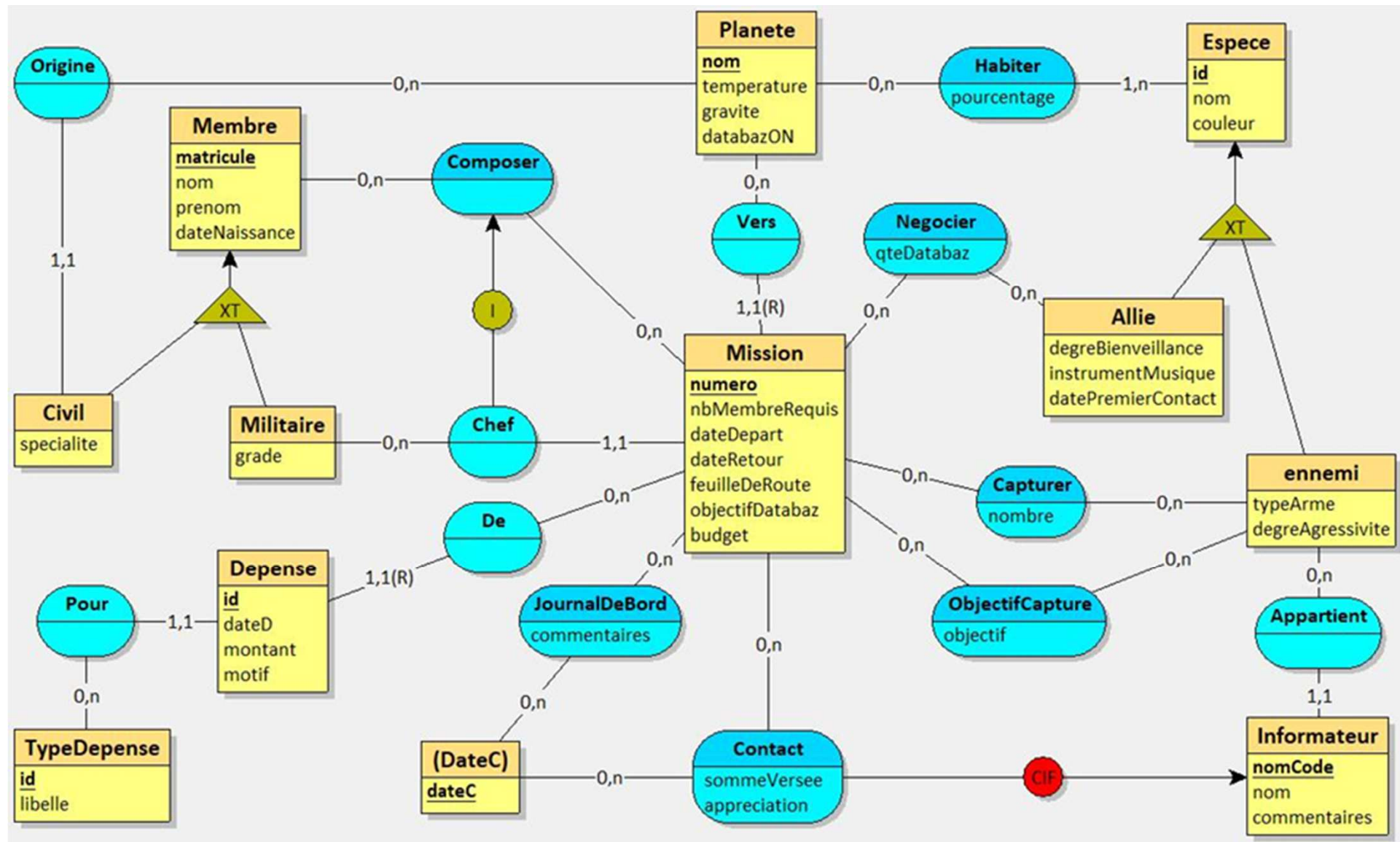


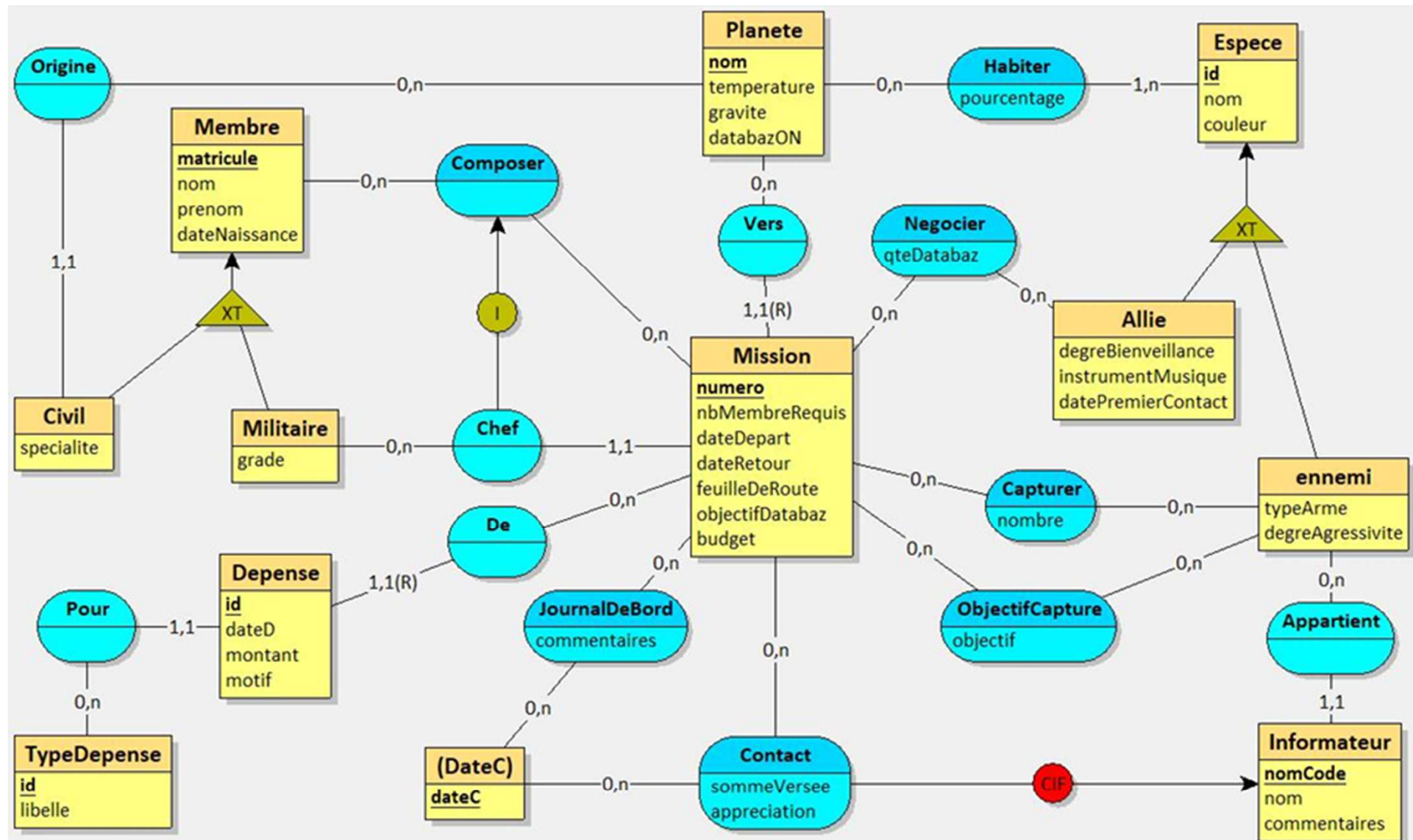


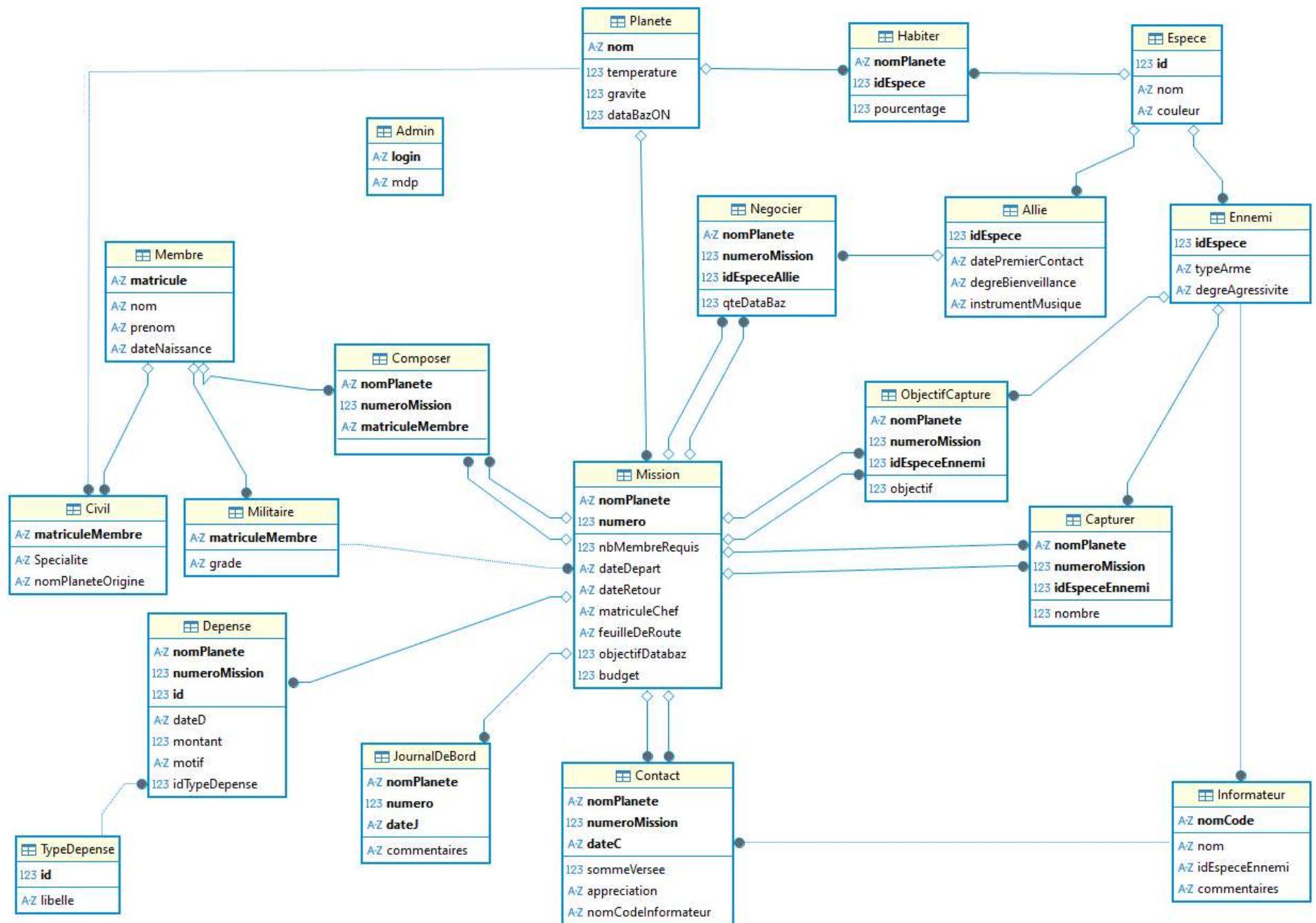


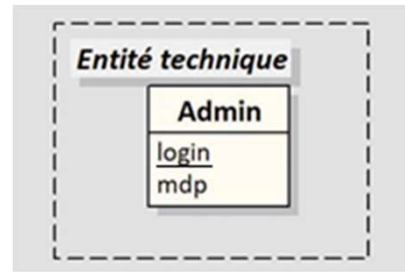












Le mot de passe est haché avec l'algorithme BCrypt et on obtient une **empreinte** que l'on va stocker dans la base de données.

	login	mdp
1	Admin	\$2y\$10\$ObY.nlop/xhmpm4ZEocrZOjD6qssrtSxm7h.dXdCyCmA5QaHNhcaK

Le hachage est une action non réversible, il est donc impossible de retrouver le mot de passe en clair à partir de l'empreinte !

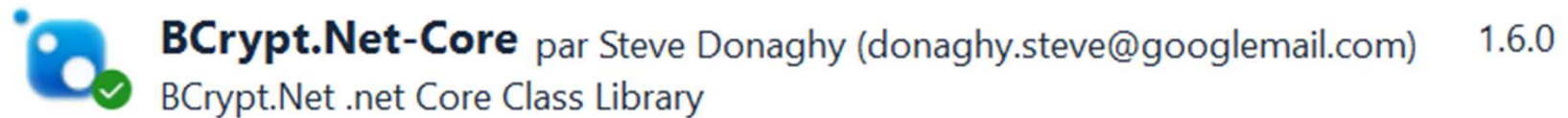
Pour vérifier que le mot de passe saisi par l'utilisateur est valide il faut comparer l'empreinte stockée dans la base avec l'empreinte générée à partir du mot de passe saisi par l'utilisateur. Pour éviter qu'un même mot de passe ne génère pas toujours la même empreinte, BCrypt met des grains de sel...

```
try
{
    //Récupération du mot de passe en fonction du login
    string sql = $"SELECT mdp FROM Admin WHERE login = '{txtLogin.Text}'";
    SQLiteCommand cmd = new SQLiteCommand(sql, Connexion.Connec);
    object resultat = cmd.ExecuteScalar();

    //Test si la requête a renvoyé un résultat non null
    if (resultat != DBNull.Value && resultat != null)
    {
        string mdpStocke = resultat.ToString();
        //Vérification du mot de passe
        bool valide = BCrypt.Net.BCrypt.Verify(txtMdp.Text, mdpStocke);

        //Résultat de l'authentification
        if (valide)
        {
            MessageBox.Show("Accès autorisé", "Authentication");
            ...
        }
        else
        {
            MessageBox.Show("Accès refusé", "Authentication");
        }
    }
}
```

Nécessite d'installer le paquet Nuget suivant :



Et mettre dans le code :

```
using Bcrypt.Net;
```

```

internal class Connexion
{
    // Objet Connection
    private static SQLiteConnection connec;

    // Constructeur privé pour empêcher l'instanciation directe depuis l'extérieur.
    private Connexion() { }

    // Méthode publique pour obtenir l'instance unique de la classe.
    public static SQLiteConnection Connec
    {
        get
        {
            // Si l'instance n'existe pas, on la crée.
            if (connec == null)
            {
                try
                {
                    // Chaîne de connexion à votre base de données
                    string chaine = @"Data Source = Stargate.bd";
                    connec = new SQLiteConnection(chaine);
                    connec.Open();
                }
                catch (SQLiteException err)
                {
                    Console.WriteLine($"Erreur lors de l'ouverture de la connexion
                }
            }
            // Dans tous les cas on renvoie la connexion
            return connec;
        }
    }
}

```

Pour appeler cette connexion :

Connexion.Connec

**Attention d'avoir le même nom
de namespace entre cette
classe et le projet !**

```
public class MesDatas
{
    private static DataSet dsGlobal = new DataSet();

    public static DataSet DsGlobal {
        get
        {
            return MesDatas.dsGlobal;
        }
    }
}
```

Pour appeler le DataSet :

MesDatas.DsGlobal

**Attention d'avoir le même nom
de namespace entre cette
classe et le projet !**